Opportunity Extension C# Script - True Line-by-Line Explanation

Customer-ready technical walkthrough for Phoenicarix-CI / ASO.Core / Opportunity

Document purpose	Explain every line of the C# script used to programmatically extend the Dynamics 365 Opportunity table.
Target environment	Phoenicarix-CI - https://phoenicarix-ci.crm4.dynamics.com
Target solution	ASO.Core, technical unique name ASOCore
Target table	Opportunity, logical name opportunity
Run method	macOS Terminal / .NET 8 console application / Microsoft.PowerPlatform.Dataverse.Client
Output	Creates missing ASO columns, skips existing columns, publishes Opportunity customizations

1. Audience-oriented summary

Grandma version: The script teaches Dynamics 365 Opportunity records new boxes to store AI, SAP, and Customer Insights information. It checks what boxes already exist, creates the missing ones, and then tells Dynamics to publish the change.

Product owner version: The script implements the Opportunity part of the ASO Phase 2 data contract. It prepares governed fields for AI recommendations, Sales Agent output, Foundry decisions, SAP context, and communication-governance state.

Developer version: The script connects to Dataverse with ServiceClient, retrieves existing Opportunity attributes, builds AttributeMetadata objects, creates missing attributes with CreateAttributeRequest, associates them to ASOCore, and publishes targeted Opportunity metadata using PublishXmlRequest.

2. Line-by-line explanation

The following table explains every numbered line in the script. Blank lines and braces are included because they matter for readability or code structure, even when they do not perform a business action by themselves.

Lines 1-45

Line	Code	What this line does	Why / how it works
1	using Microsoft.Crm.Sdk.Messages;	Imports Dataverse message classes such as PublishXmlRequest.	The script needs these SDK request classes to publish table customizations after creating metadata.
2	using Microsoft.PowerPlatform.Dataverse.Client;	Imports the ServiceClient class used to connect to Dataverse.	ServiceClient handles OAuth authentication, connection creation, and SDK request execution.
3	using Microsoft.Xrm.Sdk;	Imports core Dataverse SDK types such as Label and OrganizationRequest-related types.	These types are used throughout the script to define metadata labels and execute Dataverse operations.
4	using Microsoft.Xrm.Sdk.Messages;	Imports SDK message request/response types such as CreateAttributeRequest and RetrieveEntityRequest.	The script uses these messages to read existing columns and create new columns.
5	using Microsoft.Xrm.Sdk.Metadata;	Imports metadata classes such as StringAttributeMetadata,	These classes represent the column definitions that the

		PicklistAttributeMetadata, and OptionSetMetadata.	script creates in Dataverse.
6	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
7	const string DataverseUrl = "https://phoenicarix-ci.crm4.dynamics.com";	Stores the Dataverse environment URL for Phoenicarix-CI.	All API operations must point to the correct trial environment; this prevents accidentally creating schema in another environment.
8	const string SolutionUniqueName = "ASOCore";	Stores the technical solution unique name ASOCore.	CreateAttributeRequest uses this to place created columns into the ASO.Core solution layer instead of leaving them unmanaged in the wrong context.
9	const string EntityLogicalName = "opportunity";	Sets the target table logical name to opportunity.	Dataverse uses logical names in SDK calls; this tells the script to extend the Opportunity table.
10	const int LanguageCode = 1033;	Sets the language code to 1033, which is English.	Dataverse labels are localized; 1033 creates English labels for display names and descriptions.
11	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
12	var connectionString =	Starts creation of the Dataverse connection string.	The next lines define authentication mode, URL, client ID, and redirect URI used by ServiceClient.
13	$@"AuthType=OAuth;	Starts a verbatim interpolated connection string and sets OAuth authentication.	OAuth is the interactive sign-in method used by the script for a maker/admin user.
14	Url={DataverseUrl};	Injects the DataverseUrl constant into the connection string.	This keeps the environment URL defined once and reused safely.
15	LoginPrompt=Auto;	Allows ServiceClient to show a login prompt when needed.	This is practical on a Mac/local development machine because a browser sign-in can be launched if no token exists.
16	ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;	Uses the well-known Microsoft client ID commonly used by Dataverse tooling samples.	It enables interactive OAuth login for local SDK tooling without creating a custom app registration for this MVP script.
17	RedirectUri=http://localhost";	Sets the local redirect URI for the OAuth login flow.	After browser authentication, the token flow returns to localhost so the local process can continue.
18	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
19	using var service = new ServiceClient(connectionString);	Creates a Dataverse ServiceClient from the connection string.	This object is the main gateway used by the script to read metadata, create columns, and publish changes.
20	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
21	if (!service.IsReady)	Checks whether the Dataverse connection is usable.	The script should stop immediately if authentication or connection initialization failed.
22	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
23	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal text color to red.	Red makes connection or field-creation failures immediately visible to the operator.
24	Console.WriteLine("Connection failed:");	Prints a connection failure message.	This tells the operator that nothing should continue until authentication/connectivity is fixed.
25	Console.WriteLine(service.LastError);	Prints the detailed ServiceClient error.	The exact connection error is needed for troubleshooting authentication, URL, or permission problems.
26	Console.ResetColor();	Resets terminal text color back to normal.	Without this, all later terminal output could remain red, green, or yellow.
27	return;	Stops the script early.	This prevents schema operations from running when the Dataverse connection is not ready.
28	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
29	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
30	Console.WriteLine($"Connected to: {DataverseUrl}");	Prints the environment URL that the script connected to.	This lets the operator verify that the script is targeting Phoenicarix-CI before schema changes are made.
31	Console.WriteLine($"Target solution: {SolutionUniqueName}");	Prints the target solution unique name.	This confirms that columns should be associated with ASO.Core.
32	Console.WriteLine($"Target table: {EntityLogicalName}");	Prints the target table logical name.	This confirms that the script is extending Opportunity and not another table.
33	Console.WriteLine();	Prints an empty line in the terminal.	This separates sections of output and improves readability.
34	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
35	var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);	Retrieves the current list of Opportunity column logical names.	This enables idempotency: the script can skip fields that already exist rather than failing on duplicates.
36	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.

37	var fields = new List<AttributeMetadata>	Starts a list of column metadata definitions.	Each item in this list describes one Opportunity column that should be created.
38	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
39	Text("aso_sapcustomerid", "SAP Customer ID", 100, "SAP reference only."),	Defines a text field for the SAP Customer ID.	This keeps the ERP customer reference available on the opportunity without allowing sellers or AI agents to write directly to SAP.
40	Text("aso_saporderreference", "SAP Order Reference", 100, "Persist only after approved SAP process."),	Defines a text field for the SAP order reference.	The field is reserved for approved commercial processing; it avoids uncontrolled SAP order information being mixed into ordinary sales notes.
41	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
42	LocalChoice("aso_aiopportunitytier", "AI Opportunity Tier", new[] { "A", "B", "C" }, "AI opportunity tier."),	Defines a local choice field with A, B, and C as opportunity tier values.	The tier is specific to opportunity prioritisation, so the script keeps it local rather than creating a reusable global choice.
43	Memo("aso_aisummary", "AI Summary", "AI account/opportunity summary."),	Defines a multiline text field for an AI-generated opportunity summary.	A long text field is needed because summaries can be several sentences and should not be forced into a short text box.
44	Memo("aso_airisks", "AI Risks", "Identified risks."),	Defines a multiline text field for AI-identified risks.	Risk descriptions need room for context, evidence, and wording that a seller can understand.
45	Memo("aso_aistakeholdergaps", "AI Stakeholder Gaps", "Stakeholder coverage gaps."),	Defines a multiline text field for missing stakeholder coverage.	Complex B2B deals often require several stakeholders; this field stores the AI explanation of gaps.

Lines 46-90

Line	Code	What this line does	Why / how it works
46	Memo("aso_ainextactions", "AI Next Actions", "Recommended next actions."),	Defines a multiline text field for recommended next actions.	Next-best-actions can contain multiple steps, so multiline text is safer than a short text column.
47	LocalChoice("aso_airisklevel", "AI Risk Level", new[] { "Low", "Medium", "High" }, "AI risk level."),	Defines a local choice field with Low, Medium, and High values.	Risk level is an opportunity-specific classification and can drive views, filters, or manager review.
48	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
49	Memo("aso_aifollowupmessage", "AI Follow-up Message Draft", "Draft only; no direct customer send."),	Defines a multiline draft-message field.	The draft is explicitly not a send mechanism; Customer Insights remains the outbound communication layer.
50	DecimalNumber("aso_aiconfidence", "AI Confidence", 0m, 1m, 2, "AI confidence 0.00-1.00."),	Defines a decimal field constrained conceptually to 0.00 through 1.00.	The field stores model confidence as a normalized number suitable for thresholds and review rules.
51	DateTimeField("aso_ailastrunon", "AI Last Run On", "Last run timestamp."),	Defines a user-local date/time field for the last AI run timestamp.	This lets operations and sellers see when the AI information was last refreshed.
52	GlobalChoice("aso_aiagentstatus", "AI Agent Status", "aso_aistatus", "Uses global AI status choice."),	Defines a choice field that reuses the global AI Status choice.	Reusing the global choice keeps status values consistent across Lead, Opportunity, and later operational tables.
53	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
54	Memo("aso_aisapcommercialsummary", "AI SAP Commercial Summary", "Normalized SAP summary after governed reads."),	Defines a multiline text field for normalized SAP commercial context.	It holds safe summarized SAP context after governed reads through the intended integration boundary.
55	Memo("aso_aisapproductsummary", "AI SAP Product Summary", "Normalized SAP product context."),	Defines a multiline text field for normalized SAP product context.	Product context can be broad and descriptive, so multiline text preserves useful information.
56	Text("aso_aicorrelationid", "AI Correlation ID", 100, "Last run correlation."),	Defines a short text field for the last correlation identifier.	Correlation IDs connect Dataverse updates to Power Automate, Foundry, API, and monitoring traces.
57	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
58	GlobalChoice("aso_salesopportunityagentstatus", "Sales Opportunity Agent Status", "aso_aistatus", "Status of seller-facing opportunity agent output."),	Defines a Sales Opportunity Agent status field using the global AI status choice.	The choice is reused because the field expresses a processing/run status rather than a free-form business comment.
59	Text("aso_salesopportunityagentimportance", "Sales Opportunity Agent Importance", 100, "Native output varies by tenant."),	Defines a short text field for native agent importance output.	The guide allowed Choice/Text where native output can vary, so text avoids premature lock-in.
60	Text("aso_salesopportunityagentrisk", "Sales Opportunity Agent Risk", 100, "Native output varies by tenant."),	Defines a short text field for native agent risk output.	The exact taxonomy may differ by tenant or Sales AI configuration, so text is flexible for MVP.
61	Memo("aso_salesopportunityagentrecommendation", "Sales Opportunity Agent Recommendation", "Seller-facing recommendation."),	Defines a multiline text field for seller-facing recommendations.	Recommendations can be paragraph-length and should be readable by a seller or product owner.
62	Memo("aso_salesdealcloseagentguidance", "Sales Deal Close Agent Guidance", "Close-readiness guidance."),	Defines a multiline field for deal-close guidance.	Close guidance can include blockers, warnings, and suggested actions, so it needs rich text capacity.
63	DateTimeField("aso_salesdealcloseagentlastrunon", "Sales Deal Close Agent Last Run On", "Timestamp."),	Defines a date/time field for the last deal-close agent run.	This separates stale AI output from recently refreshed

			guidance.
64	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
65	Memo("aso_foundryfinalopportunitydecision", "Foundry Final Opportunity Decision", "Final governed orchestration decision."),	Defines a multiline field for the final governed Foundry decision.	The final orchestration decision can include reasoning and is kept flexible during MVP.
66	YesNo("aso_foundryreviewrequired", "Foundry Review Required", "Human review gate."),	Defines a Yes/No flag for human review.	A boolean flag is easy to filter and can later drive views, notifications, or approval logic.
67	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
68	GlobalChoice("aso_communicationstate", "Communication State", "aso_communicationstate", "Uses global Communication State choice."),	Defines a choice field that reuses the global Communication State choice.	The same communication state taxonomy is needed across Lead, Opportunity, Account, and Contact.
69	GlobalChoice("aso_lifecyclecommunicationstage", "Lifecycle Communication Stage", "aso_lifecyclecommunicationstage", "Uses global Lifecycle Communication Stage choice."),	Defines a choice field that reuses the global lifecycle communication stage.	This aligns Sales records with Customer Insights journey stages.
70	GlobalChoice("aso_journeyparticipationstatus", "Journey Participation Status", "aso_journeyparticipationstatus", "Uses global Journey Participation Status choice."),	Defines a choice field that reuses the global Journey Participation Status choice.	This lets the record store journey participation in a consistent governed way.
71	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
72	Text("aso_customerinsightsjourneyid", "Customer Insights Journey ID", 100, "Latest journey reference."),	Defines a text field for the Customer Insights journey identifier.	It stores the technical link/reference to the latest relevant journey.
73	Text("aso_customerinsightsjourneyname", "Customer Insights Journey Name", 200, "Latest journey name."),	Defines a text field for the Customer Insights journey name.	This is a readable reference for sellers and operations teams.
74	DateTimeField("aso_customerinsightslastentryon", "Customer Insights Last Entry On", "Last entry."),	Defines a date/time field for the latest journey entry.	This indicates when the opportunity most recently entered a journey-related process.
75	DateTimeField("aso_customerinsightslastinteractionon", "Customer Insights Last Interaction On", "Last interaction."),	Defines a date/time field for the latest Customer Insights interaction.	This supports engagement recency checks and operations troubleshooting.
76	LocalChoice("aso_customerinsightslastinteractiontype", "Customer Insights Last Interaction Type",	Defines a local choice field for interaction type values such as email sent, open, click, reply, bounce, and unsubscribe.	The values are specific to Customer Insights interaction writeback for this MVP model.
77	new[] { "EmailSent", "Open", "Click", "FormSubmit", "Reply", "Unsubscribe", "Bounce", "CustomAction" },	Provides the list of option labels for the preceding local choice field.	The options become the selectable values shown in the Dataverse choice column.
78	"Customer Insights last interaction type."),	Provides the description argument for the multi-line LocalChoice call.	This completes the Customer Insights interaction type field definition.
79	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
80	GlobalChoice("aso_emailconsentstatus", "Email Consent Status", "aso_emailconsentstatus", "Uses global Email Consent Status choice."),	Defines a choice field that reuses the global Email Consent Status choice.	Consent must be governed consistently because it controls outbound communication eligibility.
81	Text("aso_complianceprofilename", "Compliance Profile Name", 200, "Profile used."),	Defines a text field for the compliance profile name.	This stores the Customer Insights compliance profile used for the record or communication context.
82	Memo("aso_communicationholdreason", "Communication Hold Reason", "Suppression / hold reason.")	Defines a multiline text field for suppression or hold reason.	A hold reason may require free-form explanation that is unsuitable for a short code-only field.
83	};	Closes an object initializer or list initializer and ends the statement.	In this script it usually means a metadata object or the field list has been fully defined.
84	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
85	foreach (var field in fields)	Starts a loop over every planned column definition.	The script processes the field inventory one column at a time.
86	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
87	var logicalName = field.SchemaName!.ToLowerInvariant();	Reads the field schema name and normalizes it to lowercase.	Dataverse logical names are stored in lowercase, so this helps compare planned fields with existing fields reliably.
88	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
89	if (existingAttributes.Contains(logicalName))	Checks whether the target column already exists.	This prevents duplicate-column errors and allows the script to be re-run safely.
90	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.

Lines 91-135

Line	Code	What this line does	Why / how it works
91	Console.ForegroundColor = ConsoleColor.Yellow;	Changes terminal text color to yellow.	Yellow is used for skipped existing fields so the operator knows the script did not recreate them.
92	Console.WriteLine($"SKIP existing: {logicalName}");	Prints that a field was skipped because it already exists.	The operator can see that the script behaved safely instead

			of trying to recreate a column.
93	Console.ResetColor();	Resets terminal text color back to normal.	Without this, all later terminal output could remain red, green, or yellow.
94	continue;	Skips the rest of the current loop iteration.	After detecting an existing column, the script moves to the next planned field.
95	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
96	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
97	try	Starts a protected block for a field creation attempt.	If one field fails, the catch block reports the problem without hiding which field caused it.
98	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
99	Console.WriteLine($"Creating: {logicalName} ...");	Prints which column is about to be created.	This creates a readable execution log in the terminal.
100	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
101	var request = new CreateAttributeRequest	Creates the SDK request object used to create a Dataverse column.	Dataverse metadata changes are sent as requests through the organization service.
102	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
103	EntityName = EntityLogicalName,	Sets the target table for the column creation request.	Because EntityLogicalName is opportunity, the new column is created on the Opportunity table.
104	Attribute = field,	Attaches the current column metadata definition to the request.	This tells Dataverse exactly what column type, schema name, label, description, and settings to create.
105	SolutionUniqueName = SolutionUniqueName	Associates the created column with the ASO.Core solution.	This is critical for ALM because the new column must be part of the project solution.
106	};	Closes an object initializer or list initializer and ends the statement.	In this script it usually means a metadata object or the field list has been fully defined.
107	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
108	service.Execute(request);	Sends the create-column request to Dataverse.	This is the line that actually asks Dataverse to create the metadata column.
109	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
110	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal text color to green.	Green makes successful creation and completion messages easy to recognize.
111	Console.WriteLine($"CREATED: {logicalName}");	Prints a successful creation message for the current field.	The green output confirms the column was created successfully.
112	Console.ResetColor();	Resets terminal text color back to normal.	Without this, all later terminal output could remain red, green, or yellow.
113	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
114	catch (Exception ex)	Starts error handling for a failed field creation attempt.	Failures are captured and printed instead of crashing without context.
115	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
116	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal text color to red.	Red makes connection or field-creation failures immediately visible to the operator.
117	Console.WriteLine($"FAILED: {logicalName}");	Prints which field failed.	This identifies the exact column requiring investigation.
118	Console.WriteLine(ex.Message);	Prints the exception message from Dataverse or the SDK.	The exact error usually explains missing choices, duplicate names, permissions, or invalid metadata settings.
119	Console.ResetColor();	Resets terminal text color back to normal.	Without this, all later terminal output could remain red, green, or yellow.
120	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
121	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
122	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
123	Console.WriteLine();	Prints an empty line in the terminal.	This separates sections of output and improves readability.
124	Console.WriteLine("Publishing Opportunity table customizations...");	Prints that the script is about to publish Opportunity customizations.	Publishing makes newly created metadata visible/usable in the maker and model-driven app experience.
125	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
126	service.Execute(new PublishXmlRequest	Starts a Dataverse publish request.	After metadata creation, publishing is needed so the table

			changes become active.
127	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
128	ParameterXml = "<importexportxml><entities><entity>opportunity</entity></entities></importexportxml>"	Sets the publish XML payload.	The XML tells Dataverse exactly which table customizations to publish.
129	});	Completes a C# statement.	This line finalizes a variable assignment, method call, return statement, or object construction.
130	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
131	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal text color to green.	Green makes successful creation and completion messages easy to recognize.
132	Console.WriteLine("Done. Validate in ASO.Core → Tables → Opportunity → Columns.");	Prints the final success message and validation instruction.	The operator is told to check ASO.Core -> Tables -> Opportunity -> Columns after execution.
133	Console.ResetColor();	Resets terminal text color back to normal.	Without this, all later terminal output could remain red, green, or yellow.
134	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
135	static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)	Declares a helper method that returns all existing column logical names for a table.	The method supports re-runnable, safe behavior by allowing duplicate checks before creation.

Lines 136-180

Line	Code	What this line does	Why / how it works
136	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
137	var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest	Executes a retrieve-entity metadata request and casts the response.	This retrieves table metadata, including its existing attributes/columns.
138	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
139	LogicalName = entityLogicalName,	Sets the table logical name for the retrieve request.	This makes the helper reusable for any table, although this script calls it for Opportunity.
140	EntityFilters = EntityFilters.Attributes,	Requests only attribute/column metadata.	This keeps the metadata request focused on the information needed for duplicate detection.
141	RetrieveAsIfPublished = true	Includes unpublished metadata in the result.	This is important because recently created or unpublished columns should still be detected as existing.
142	});	Completes a C# statement.	This line finalizes a variable assignment, method call, return statement, or object construction.
143	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
144	return response.EntityMetadata.Attributes	Starts returning the table attribute collection as a processed set.	The next chained lines filter and transform the attributes into logical names.
145	.Where(a => !string.IsNullOrWhiteSpace(a.LogicalName))	Filters out attributes without a logical name.	This avoids null or blank names entering the duplicate-check set.
146	.Select(a => a.LogicalName!)	Selects only the logical name from each attribute.	The script only needs names to check whether a field already exists.
147	.ToHashSet(StringComparer.OrdinalIgnoreCase);	Converts the names into a case-insensitive hash set.	A hash set makes lookups fast and avoids false mismatches caused by uppercase/lowercase differences.
148	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
149	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
150	static Label Label(string value) => new(value, LanguageCode);	Declares a helper that creates a Dataverse Label object.	Display names and descriptions require localized Dataverse labels, not plain strings.
151	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
152	static AttributeRequiredLevelManagedProperty Optional()	Declares a helper for optional required-level metadata.	All created fields are optional for the MVP so existing records do not break and users are not forced to populate system-owned fields.
153	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
154	return new AttributeRequiredLevelManagedProperty(AttributeRequiredLev	Returns the Dataverse optional required-level object.	This standardizes required-level handling for all helper methods.

	el.None);		
155	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
156	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
157	static StringAttributeMetadata Text(string schemaName, string displayName, int maxLength, string description)	Declares a helper to build single-line text column metadata.	It prevents repeated boilerplate whenever the script creates a text field.
158	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
159	return new StringAttributeMetadata	Starts creating metadata for a Dataverse single-line text column.	This object type tells Dataverse the new field should store short text values.
160	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
161	SchemaName = schemaName,	Sets the column schema name from the helper parameter.	This controls the technical Dataverse name such as aso_sapcustomerid.
162	DisplayName = Label(displayName),	Sets the user-facing display name.	This is the readable label shown to makers and users in Power Apps.
163	Description = Label(description),	Sets the column description.	Descriptions help IT teams and makers understand the purpose of the field later.
164	RequiredLevel = Optional(),	Sets the field as not business-required.	This keeps the MVP safe for existing records and prevents forced input on system-owned AI/integration fields.
165	MaxLength = maxLength	Sets the maximum length for a text field.	This limits the stored text size according to the field definition passed into the helper.
166	};	Closes an object initializer or list initializer and ends the statement.	In this script it usually means a metadata object or the field list has been fully defined.
167	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
168	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
169	static MemoAttributeMetadata Memo(string schemaName, string displayName, string description)	Declares a helper to build multiline text column metadata.	It is used for longer AI outputs, explanations, summaries, and rationale fields.
170	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
171	return new MemoAttributeMetadata	Starts creating metadata for a Dataverse multiline text column.	Memo fields are appropriate when the content may exceed a short text value.
172	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
173	SchemaName = schemaName,	Sets the column schema name from the helper parameter.	This controls the technical Dataverse name such as aso_sapcustomerid.
174	DisplayName = Label(displayName),	Sets the user-facing display name.	This is the readable label shown to makers and users in Power Apps.
175	Description = Label(description),	Sets the column description.	Descriptions help IT teams and makers understand the purpose of the field later.
176	RequiredLevel = Optional(),	Sets the field as not business-required.	This keeps the MVP safe for existing records and prevents forced input on system-owned AI/integration fields.
177	MaxLength = 4000	Sets multiline text capacity to 4000 characters.	This gives AI summaries, risks, guidance, and hold reasons enough room for useful narrative content.
178	};	Closes an object initializer or list initializer and ends the statement.	In this script it usually means a metadata object or the field list has been fully defined.
179	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
180	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.

Lines 181-225

Line	Code	What this line does	Why / how it works
181	static DecimalAttributeMetadata DecimalNumber(string schemaName, string displayName, decimal min, decimal max, int precision, string description)	Declares a helper to build decimal number column metadata.	The Opportunity AI Confidence field needs decimal precision rather than integer values.
182	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.

183	return new DecimalAttributeMetadata	Starts creating metadata for a Dataverse decimal field.	This metadata type stores values like 0.85 rather than plain text.
184	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
185	SchemaName = schemaName,	Sets the column schema name from the helper parameter.	This controls the technical Dataverse name such as aso_sapcustomerid.
186	DisplayName = Label(displayName),	Sets the user-facing display name.	This is the readable label shown to makers and users in Power Apps.
187	Description = Label(description),	Sets the column description.	Descriptions help IT teams and makers understand the purpose of the field later.
188	RequiredLevel = Optional(),	Sets the field as not business-required.	This keeps the MVP safe for existing records and prevents forced input on system-owned AI/integration fields.
189	MinValue = min,	Sets the minimum allowed decimal value.	For AI confidence, this supports the normalized 0.00 lower bound.
190	MaxValue = max,	Sets the maximum allowed decimal value.	For AI confidence, this supports the normalized 1.00 upper bound.
191	Precision = precision	Sets the number of decimal places.	Precision 2 is enough for values such as 0.75 or 0.93.
192	};	Closes an object initializer or list initializer and ends the statement.	In this script it usually means a metadata object or the field list has been fully defined.
193	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
194	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
195	static DateTimeAttributeMetadata DateTimeField(string schemaName, string displayName, string description)	Declares a helper to build date/time column metadata.	The script uses it for AI run timestamps and Customer Insights interaction timestamps.
196	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
197	return new DateTimeAttributeMetadata	Starts creating metadata for a Dataverse date/time field.	This metadata type stores timestamps rather than plain text.
198	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
199	SchemaName = schemaName,	Sets the column schema name from the helper parameter.	This controls the technical Dataverse name such as aso_sapcustomerid.
200	DisplayName = Label(displayName),	Sets the user-facing display name.	This is the readable label shown to makers and users in Power Apps.
201	Description = Label(description),	Sets the column description.	Descriptions help IT teams and makers understand the purpose of the field later.
202	RequiredLevel = Optional(),	Sets the field as not business-required.	This keeps the MVP safe for existing records and prevents forced input on system-owned AI/integration fields.
203	Format = DateTimeFormat.DateAndTime,	Configures the field to store both date and time.	Operational run timestamps require the time of day, not only the calendar date.
204	DateTimeBehavior = DateTimeBehavior.UserLocal	Sets the date/time behavior to user-local.	The timestamp displays in the viewer's local timezone, matching typical model-driven app behavior.
205	};	Closes an object initializer or list initializer and ends the statement.	In this script it usually means a metadata object or the field list has been fully defined.
206	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
207	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
208	static BooleanAttributeMetadata YesNo(string schemaName, string displayName, string description)	Declares a helper to build Yes/No column metadata.	It is used for boolean flags such as Foundry Review Required.
209	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
210	return new BooleanAttributeMetadata	Starts creating metadata for a Dataverse Yes/No field.	Boolean metadata stores true/false style information.
211	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
212	SchemaName = schemaName,	Sets the column schema name from the helper parameter.	This controls the technical Dataverse name such as aso_sapcustomerid.
213	DisplayName = Label(displayName),	Sets the user-facing display name.	This is the readable label shown to makers and users in Power Apps.
214	Description = Label(description),	Sets the column description.	Descriptions help IT teams and makers understand the purpose of the field later.

215	RequiredLevel = Optional(),	Sets the field as not business-required.	This keeps the MVP safe for existing records and prevents forced input on system-owned AI/integration fields.
216	DefaultValue = false,	Sets the default value to No/false.	New opportunities should not require review unless automation or a user explicitly marks them as requiring review.
217	OptionSet = new BooleanOptionSetMetadata(	Defines the Yes and No labels for the boolean field.	Dataverse Yes/No fields still have labels that users see in the UI.
218	new OptionMetadata(Label("Yes"), 1),	Defines the Yes option with value 1.	This is the true value displayed as Yes in the app.
219	new OptionMetadata(Label("No"), 0)	Defines the No option with value 0.	This is the false value displayed as No in the app.
220	)	Closes a method call or constructor call.	This completes a grouped expression that started on a previous line.
221	};	Closes an object initializer or list initializer and ends the statement.	In this script it usually means a metadata object or the field list has been fully defined.
222	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
223	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
224	static PicklistAttributeMetadata LocalChoice(string schemaName, string displayName, string[] values, string description)	Declares a helper to build a local choice column.	Local choices are used when values belong only to one specific field and do not need global reuse.
225	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.

Lines 226-262

Line	Code	What this line does	Why / how it works
226	var optionSet = new OptionSetMetadata	Creates an option set metadata object.	The next lines configure whether the choice is local/global and what type of choice it is.
227	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
228	IsGlobal = false,	Marks the option set as local.	The choice values will belong only to this column instead of becoming a reusable global choice component.
229	OptionSetType = OptionSetType.Picklist	Marks the choice as a single-select picklist.	Users will pick one value from a defined list.
230	};	Closes an object initializer or list initializer and ends the statement.	In this script it usually means a metadata object or the field list has been fully defined.
231	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
232	foreach (var value in values)	Loops through every option label passed into the helper.	This creates one Dataverse option for each value such as A, B, or C.
233	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
234	optionSet.Options.Add(new OptionMetadata(Label(value), null));	Adds a new option to the choice field.	The label becomes a selectable value in the Dataverse choice column; null lets Dataverse assign the numeric option value.
235	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
236	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
237	return new PicklistAttributeMetadata	Starts creating metadata for a Dataverse choice column.	PicklistAttributeMetadata is the SDK type for single-select choice fields.
238	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
239	SchemaName = schemaName,	Sets the column schema name from the helper parameter.	This controls the technical Dataverse name such as aso_sapcustomerid.
240	DisplayName = Label(displayName),	Sets the user-facing display name.	This is the readable label shown to makers and users in Power Apps.
241	Description = Label(description),	Sets the column description.	Descriptions help IT teams and makers understand the purpose of the field later.
242	RequiredLevel = Optional(),	Sets the field as not business-required.	This keeps the MVP safe for existing records and prevents forced input on system-owned AI/integration fields.
243	OptionSet = optionSet	Attaches the locally built option set to the choice column.	This completes the local choice field definition before it is sent to Dataverse.
244	};	Closes an object initializer or list initializer and ends the	In this script it usually means a metadata object or the field

		statement.	list has been fully defined.
245	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
246	<blank>	Blank spacer line.	It improves readability only; it has no runtime effect.
247	static PicklistAttributeMetadata GlobalChoice(string schemaName, string displayName, string globalChoiceName, string description)	Declares a helper to build a choice column that reuses an existing global choice.	Global choices keep shared values consistent across multiple tables.
248	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
249	return new PicklistAttributeMetadata	Starts creating metadata for a Dataverse choice column.	PicklistAttributeMetadata is the SDK type for single-select choice fields.
250	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
251	SchemaName = schemaName,	Sets the column schema name from the helper parameter.	This controls the technical Dataverse name such as aso_sapcustomerid.
252	DisplayName = Label(displayName),	Sets the user-facing display name.	This is the readable label shown to makers and users in Power Apps.
253	Description = Label(description),	Sets the column description.	Descriptions help IT teams and makers understand the purpose of the field later.
254	RequiredLevel = Optional(),	Sets the field as not business-required.	This keeps the MVP safe for existing records and prevents forced input on system-owned AI/integration fields.
255	OptionSet = new OptionSetMetadata	Starts an inline global option set reference.	The nested settings identify which global choice the column should use.
256	{	Opens a code block.	A block groups the following statements under the preceding statement, method, loop, or object initializer.
257	IsGlobal = true,	Marks the option set reference as global.	Dataverse will link this column to an existing reusable choice instead of creating local values.
258	Name = globalChoiceName,	Sets the technical name of the global choice to reuse.	This must match the existing global choice name in ASO.Core, such as aso_aistatus.
259	OptionSetType = OptionSetType.Picklist	Marks the choice as a single-select picklist.	Users will pick one value from a defined list.
260	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.
261	};	Closes an object initializer or list initializer and ends the statement.	In this script it usually means a metadata object or the field list has been fully defined.
262	}	Closes a code block.	This ends the current method, loop, conditional block, or initializer scope.

3. Full script appendix

This appendix contains the full C# script in one block for copy/paste validation.

```
using Microsoft.Crm.Sdk.Messages;
using Microsoft.PowerPlatform.Dataverse.Client;
using Microsoft.Xrm.Sdk;
using Microsoft.Xrm.Sdk.Messages;
using Microsoft.Xrm.Sdk.Metadata;

const string DataverseUrl = "https://phoenicarix-ci.crm4.dynamics.com";
const string SolutionUniqueName = "ASOCore";
const string EntityLogicalName = "opportunity";
const int LanguageCode = 1033;

var connectionString =
   $@"AuthType=OAuth;
     Url={DataverseUrl};
     LoginPrompt=Auto;
     ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;
     RedirectUri=http://localhost";

using var service = new ServiceClient(connectionString);

if (!service.IsReady)
{
   Console.ForegroundColor = ConsoleColor.Red;
   Console.WriteLine("Connection failed:");
   Console.WriteLine(service.LastError);
   Console.ResetColor();
   return;
}

Console.WriteLine($"Connected to: {DataverseUrl}");
Console.WriteLine($"Target solution: {SolutionUniqueName}");
Console.WriteLine($"Target table: {EntityLogicalName}");
Console.WriteLine();

var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);

var fields = new List<AttributeMetadata>
{
   Text("aso_sapcustomerid", "SAP Customer ID", 100, "SAP reference only."),
   Text("aso_saporderreference", "SAP Order Reference", 100, "Persist only after approved SAP process."),

   LocalChoice("aso_aiopportunitytier", "AI Opportunity Tier", new[] { "A", "B", "C" }, "AI opportunity tier."),
   Memo("aso_aisummary", "AI Summary", "AI account/opportunity summary."),
   Memo("aso_airisks", "AI Risks", "Identified risks."),
   Memo("aso_aistakeholdergaps", "AI Stakeholder Gaps", "Stakeholder coverage gaps."),
   Memo("aso_ainextactions", "AI Next Actions", "Recommended next actions."),
   LocalChoice("aso_airisklevel", "AI Risk Level", new[] { "Low", "Medium", "High" }, "AI risk level."),
```

```
    Memo("aso_aifollowupmessage", "AI Follow-up Message Draft", "Draft only; no direct customer send."),
    DecimalNumber("aso_aiconfidence", "AI Confidence", 0m, 1m, 2, "AI confidence 0.00-1.00."),
    DateTimeField("aso_ailastrunon", "AI Last Run On", "Last run timestamp."),
    GlobalChoice("aso_aiagentstatus", "AI Agent Status", "aso_aistatus", "Uses global AI status choice."),

    Memo("aso_aisapcommercialsummary", "AI SAP Commercial Summary", "Normalized SAP summary after governed reads."),
    Memo("aso_aisapproductsummary", "AI SAP Product Summary", "Normalized SAP product context."),
    Text("aso_aicorrelationid", "AI Correlation ID", 100, "Last run correlation."),

    GlobalChoice("aso_salesopportunityagentstatus", "Sales Opportunity Agent Status", "aso_aistatus", "Status of seller-facing opportunity agent output."),
    Text("aso_salesopportunityagentimportance", "Sales Opportunity Agent Importance", 100, "Native output varies by tenant."),
    Text("aso_salesopportunityagentrisk", "Sales Opportunity Agent Risk", 100, "Native output varies by tenant."),
    Memo("aso_salesopportunityagentrecommendation", "Sales Opportunity Agent Recommendation", "Seller-facing recommendation."),
    Memo("aso_salesdealcloseagentguidance", "Sales Deal Close Agent Guidance", "Close-readiness guidance."),
    DateTimeField("aso_salesdealcloseagentlastrunon", "Sales Deal Close Agent Last Run On", "Timestamp."),

    Memo("aso_foundryfinalopportunitydecision", "Foundry Final Opportunity Decision", "Final governed orchestration decision."),
    YesNo("aso_foundryreviewrequired", "Foundry Review Required", "Human review gate."),

    GlobalChoice("aso_communicationstate", "Communication State", "aso_communicationstate", "Uses global Communication State choice."),
    GlobalChoice("aso_lifecyclecommunicationstage", "Lifecycle Communication Stage", "aso_lifecyclecommunicationstage", "Uses global Lifecycle Communication Stage choice."),
    GlobalChoice("aso_journeyparticipationstatus", "Journey Participation Status", "aso_journeyparticipationstatus", "Uses global Journey Participation Status choice."),

    Text("aso_customerinsightsjourneyid", "Customer Insights Journey ID", 100, "Latest journey reference."),
    Text("aso_customerinsightsjourneyname", "Customer Insights Journey Name", 200, "Latest journey name."),
    DateTimeField("aso_customerinsightslastentryon", "Customer Insights Last Entry On", "Last entry."),
    DateTimeField("aso_customerinsightslastinteractionon", "Customer Insights Last Interaction On", "Last interaction."),
    LocalChoice("aso_customerinsightslastinteractiontype", "Customer Insights Last Interaction Type",
        new[] { "EmailSent", "Open", "Click", "FormSubmit", "Reply", "Unsubscribe", "Bounce", "CustomAction" },
        "Customer Insights last interaction type."),

    GlobalChoice("aso_emailconsentstatus", "Email Consent Status", "aso_emailconsentstatus", "Uses global Email Consent Status choice."),
    Text("aso_complianceprofilename", "Compliance Profile Name", 200, "Profile used."),
    Memo("aso_communicationholdreason", "Communication Hold Reason", "Suppression / hold reason.")
};

foreach (var field in fields)
{
    var logicalName = field.SchemaName!.ToLowerInvariant();

    if (existingAttributes.Contains(logicalName))
    {
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine($"SKIP existing: {logicalName}");
        Console.ResetColor();
        continue;
    }

    try
    {
        Console.WriteLine($"Creating: {logicalName} ...");

        var request = new CreateAttributeRequest
```

```
        {
            EntityName = EntityLogicalName,
            Attribute = field,
            SolutionUniqueName = SolutionUniqueName
        };

        service.Execute(request);

        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine($"CREATED: {logicalName}");
        Console.ResetColor();
    }
    catch (Exception ex)
    {
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine($"FAILED: {logicalName}");
        Console.WriteLine(ex.Message);
        Console.ResetColor();
    }
}

Console.WriteLine();
Console.WriteLine("Publishing Opportunity table customizations...");

service.Execute(new PublishXmlRequest
{
    ParameterXml = "<importexportxml><entities><entity>opportunity</entity></entities></importexportxml>"
});

Console.ForegroundColor = ConsoleColor.Green;
Console.WriteLine("Done. Validate in ASO.Core → Tables → Opportunity → Columns.");
Console.ResetColor();

static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)
{
    var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest
    {
        LogicalName = entityLogicalName,
        EntityFilters = EntityFilters.Attributes,
        RetrieveAsIfPublished = true
    });

    return response.EntityMetadata.Attributes
        .Where(a => !string.IsNullOrWhiteSpace(a.LogicalName))
        .Select(a => a.LogicalName!)
        .ToHashSet(StringComparer.OrdinalIgnoreCase);
}

static Label Label(string value) => new(value, LanguageCode);

static AttributeRequiredLevelManagedProperty Optional()
{
    return new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.None);
```

```
}

static StringAttributeMetadata Text(string schemaName, string displayName, int maxLength, string description)
{
    return new StringAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        MaxLength = maxLength
    };
}

static MemoAttributeMetadata Memo(string schemaName, string displayName, string description)
{
    return new MemoAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        MaxLength = 4000
    };
}

static DecimalAttributeMetadata DecimalNumber(string schemaName, string displayName, decimal min, decimal max, int precision, string description)
{
    return new DecimalAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        MinValue = min,
        MaxValue = max,
        Precision = precision
    };
}

static DateTimeAttributeMetadata DateTimeField(string schemaName, string displayName, string description)
{
    return new DateTimeAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        Format = DateTimeFormat.DateAndTime,
        DateTimeBehavior = DateTimeBehavior.UserLocal
    };
}
```

```
static BooleanAttributeMetadata YesNo(string schemaName, string displayName, string description)
{
    return new BooleanAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        DefaultValue = false,
        OptionSet = new BooleanOptionSetMetadata(
            new OptionMetadata(Label("Yes"), 1),
            new OptionMetadata(Label("No"), 0)
        )
    };
}

static PicklistAttributeMetadata LocalChoice(string schemaName, string displayName, string[] values, string description)
{
    var optionSet = new OptionSetMetadata
    {
        IsGlobal = false,
        OptionSetType = OptionSetType.Picklist
    };

    foreach (var value in values)
    {
        optionSet.Options.Add(new OptionMetadata(Label(value), null));
    }

    return new PicklistAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        OptionSet = optionSet
    };
}

static PicklistAttributeMetadata GlobalChoice(string schemaName, string displayName, string globalChoiceName, string description)
{
    return new PicklistAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        OptionSet = new OptionSetMetadata
        {
            IsGlobal = true,
            Name = globalChoiceName,
            OptionSetType = OptionSetType.Picklist
        }
```

```
    };
}
```